

NEU – DS&AI

Data Structures and Algorithms in Python

Lecture 4: Sorting, Selection and Searching Algorithms

Faculty of Data Science and Artificial Intelligence

National Economics University

Lecture 4, Part 1: Table of Contents

- 1 Why Study Sorting Algorithms?
- 2 Insertion Sort and Bubble Sort
- 3 Merge Sort and Quick Sort
- 4 Counting Sort and Radix Sort
- 5 Comparing Sorting Algorithms
- 6 Python's Built-In Sorting Functions

By the end of this lecture, students should be able to:

- ① Understand why sorting is a fundamental algorithmic operation.
- ② Understand insertion sort, merge sort, bubble sort, quicksort, counting sort, and radix sort.
- ③ Write language-independent pseudocode for comparison-based and integer-key sorting algorithms.
- ④ Compare their time, auxiliary-space, stability, adaptiveness, and input-domain assumptions.
- ⑤ Use Python's built-in sorting functions correctly with keys.

SECTION 1

Why Study Sorting Algorithms?

Sorting is everywhere

\$

Laptop A: \$800
Laptop B: \$600

Online shopping

Sort by price or rating

09:30	+500
10:15	-200
13:40	+900

Transactions

Sort by time or amount

1. Minh – 3.9
2. Lan – 3.7
3. An – 3.5

Student ranking

Sort by GPA

VN123 – 09:10
VN205 – 09:35
VN310 – 10:05

Flight schedule

Sort by departure time

What is common in all these examples?

The same data becomes much easier to **compare, search, and use** when it is placed in a meaningful order.

Why does sorting help?

Unsorted data

[42, 17, 8, 33, 21, 50, 12]

Find 33

Where should we look first?



Sorted data

[8, 12, 17, 21, 33, 42, 50]

Find 33

Now the order gives us information.

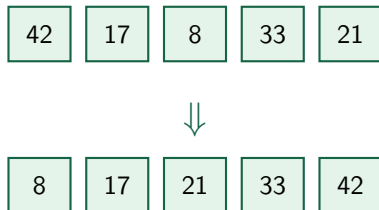
Sorting is often not the final goal

We usually sort data because the **next operation** becomes easier: searching, ranking, detecting duplicates, finding extremes, or reporting.

What is sorting?

Sorting

Sorting rearranges a collection of records according to a specified **key** and **order**.



$$k(A[0]) \leq k(A[1]) \leq \dots \leq k(A[n-1])$$

But there is more than one way to create this order

Different sorting algorithms use very different strategies.

The sorting problem must be specified precisely

Problem specification

Given records $A[0..n-1]$ and a key function k , rearrange the records so that

$$k(A[0]) \leq k(A[1]) \leq \dots \leq k(A[n-1]).$$

Correctness requirements

- The output is ordered by the chosen key.
- No input record is lost or duplicated.

Design questions

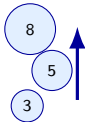
Is the sort stable? In-place? Adaptive? What are its best, average, and worst cases?

We already use sorting strategies in real life



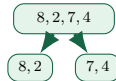
Insertion Sort

Insert a new card into an already sorted hand.



Bubble Sort

Large values gradually “bubble” toward the end.



Merge Sort

Divide into smaller parts, sort them, then merge.



Quick Sort

Choose a pivot and split values into two groups.

The important question

How does each strategy create order, and how much work does it require?

Vocabulary for comparing sorting algorithms

Property	Meaning
Stable	Equal-key records keep their original relative order.
In-place	Uses only a small amount of auxiliary memory.
Adaptive	Runs faster when the input is already partly sorted.
Comparison sort	Learns the order by comparing pairs of keys.
Integer-key sort	Uses keys as array indices or digits, not only pairwise comparisons.
Online	Can place new items as they arrive, without seeing all input.

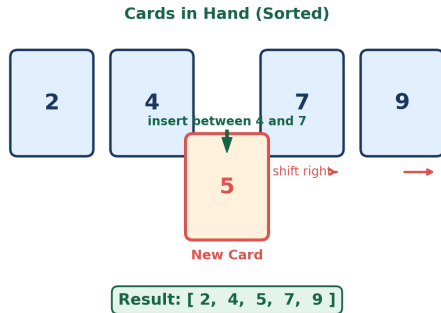
Running example

We will repeatedly sort $[7, 3, 5, 2, 6, 4]$ and count comparisons, movements, recursion, and extra storage.

SECTION 2

Insertion Sort and Bubble Sort

Insertion sort: like arranging cards in your hand



What would you do?

You already hold

2, 4, 7, 9.

A new card, 5, arrives.

Natural strategy

- 1 Compare 5 with the cards.
- 2 Move larger cards to the right.
- 3 Insert 5 between 4 and 7.

That is the idea behind Insertion Sort.

Trace insertion sort

Pass	Key	Sequence after insertion
Start	–	[7, 3, 5, 2, 6, 4]
1	3	[3, 7, 5, 2, 6, 4]
2	5	[3, 5, 7, 2, 6, 4]
3	2	[2, 3, 5, 7, 6, 4]
4	6	[2, 3, 5, 6, 7, 4]
5	4	[2, 3, 4, 5, 6, 7]

Trace invariant

At the end of pass i , the prefix $A[0..i]$ is sorted and contains exactly the original records from that prefix.

Insertion-sort pseudocode

```
1  FUNCTION INSERTION_SORT(A)
2  INPUT: array A of length n
3  OUTPUT: A sorted in non-decreasing order
4  FOR i = 1 TO n - 1 DO
5      key = A[i]
6      j = i - 1
7      WHILE j >= 0 AND A[j] > key DO
8          A[j + 1] = A[j]
9          j = j - 1
10     A[j + 1] = key
11 RETURN A
```

What costs time?

Comparisons with key and shifts of larger elements.

Why stable?

Equal items are not shifted past each other because the test uses $>$ rather than $>=$.

Insertion sort: correctness and complexity

Correctness argument

- 1 Before a pass, the prefix is sorted.
- 2 Shifting opens the correct position for the key.
- 3 Inserting the key preserves order and all records.
- 4 After the final pass, the whole array is sorted.

Performance

- Best case: $\Theta(n)$
- Average case: $\Theta(n^2)$
- Worst case: $\Theta(n^2)$
- Auxiliary space: $\Theta(1)$

Best use

Small inputs or inputs that are already nearly sorted.

Activity: predict insertion-sort work

Input A

[1, 2, 3, 4, 5, 6]

Input B

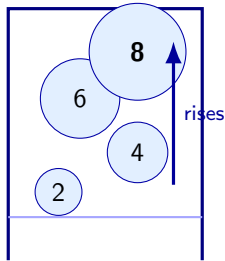
[6, 5, 4, 3, 2, 1]

Student task

For each input:

- 1 Count the successful shifts.
- 2 Identify the best or worst case.
- 3 Explain why the two inputs produce different work.

Why is it called Bubble Sort?



Think about bubbles in water

Larger bubbles rise toward the surface.

Bubble Sort uses a similar visual idea

Adjacent values are repeatedly compared.

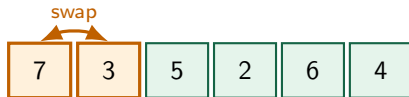
If they are in the wrong order, they swap.

As comparisons continue, a large value moves step by step toward the end of the array.

After one complete pass

The largest remaining value has reached its final position.

From the bubble idea to an array



[7, 3, 5, 2, 6, 4]

⇓

[3, 7, 5, 2, 6, 4]

⇓

[3, 5, 7, 2, 6, 4]

⇓ ... ⇓

[3, 5, 2, 6, 4, 7]

One pass

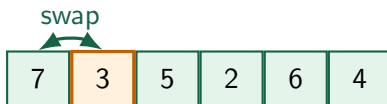
The value 7 does not jump directly to the end. It moves there through a sequence of **adjacent comparisons and swaps**.

FDA

Bubble sort: repeatedly move the largest value right

One pass

Compare adjacent values. Swap them when they are out of order. After one full pass, the largest remaining value reaches its final position.



After the first complete pass

[3, 5, 2, 6, 4, 7]; the final 7 never needs to move again.

Trace bubble sort by passes

Pass	Fixed suffix	Sequence after pass
Start	none	[7, 3, 5, 2, 6, 4]
1	[7]	[3, 5, 2, 6, 4, 7]
2	[6, 7]	[3, 2, 5, 4, 6, 7]
3	[5, 6, 7]	[2, 3, 4, 5, 6, 7]
4	no swaps	stop early

Pass invariant

After each pass, the largest value in the unsorted prefix has moved to its final position.

Optimized bubble-sort pseudocode

```
1 FUNCTION BUBBLE_SORT(A)
2 INPUT: array A of length n
3 OUTPUT: A sorted in non-decreasing order
4   FOR end = LENGTH(A) - 1 DOWNTO 1 DO
5     swapped = FALSE
6     FOR i = 0 TO end - 1 DO
7       IF A[i] > A[i + 1] THEN
8         SWAP A[i], A[i + 1]
9         swapped = TRUE
10    IF swapped == FALSE THEN
11      RETURN A
12  RETURN A
```

Performance

Best: $\Theta(n)$ with early stopping.
Average/worst: $\Theta(n^2)$.
Space: $\Theta(1)$.

Teaching value

Simple to visualize, but rarely the best general-purpose choice.

Activity: insertion sort or bubble sort?

Input

[1, 2, 3, 5, 4, 6, 7]

Work to record

Count comparisons, shifts, and swaps for both algorithms.

Discussion

- 1 Which algorithm moves fewer records?
- 2 Where does early stopping help?
- 3 Which invariant is easier to use when tracing?

SECTION 3

Merge Sort and Quick Sort

Merge sort: merge two already sorted lines

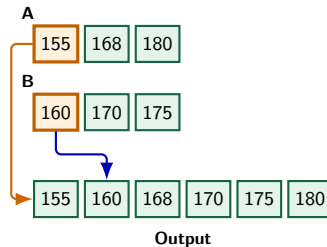
Imagine two sorted queues

$A = [155, 168, 180]$

$B = [160, 170, 175]$

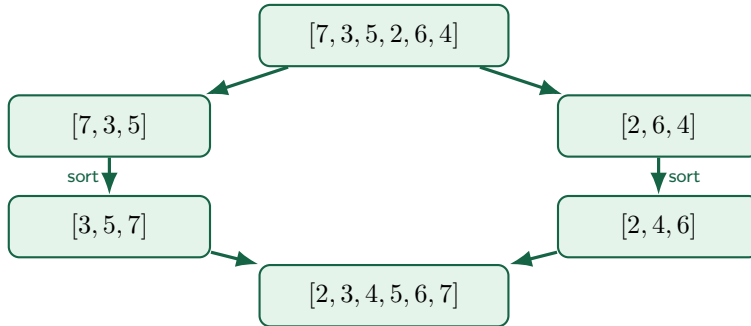
How do we merge them?

Compare the **first remaining values**.
Move the smaller one to the output,
then repeat.



Merge step: compare the two front values and repeatedly take the smaller one.

Merge sort: divide, solve, combine



Key insight

Merging two sorted sequences is linear; merge sort creates such sequences recursively.

Trace the merge operation

Merge $L = [3, 5, 7]$ and $R = [2, 4, 6]$

Compare only the first unmerged value in each sequence.

Compare	Take	Output so far
3 and 2	2	[2]
3 and 4	3	[2, 3]
5 and 4	4	[2, 3, 4]
5 and 6	5	[2, 3, 4, 5]
7 and 6	6	[2, 3, 4, 5, 6]
right empty	7	[2, 3, 4, 5, 6, 7]

Merge pseudocode

```
FUNCTION MERGE(L, R)
INPUT: sorted lists L and R
OUTPUT: one sorted list containing all values
1   result = empty list
2   i = 0; j = 0
3   WHILE i < LENGTH(L) AND j < LENGTH(R) DO
4       IF L[i] <= R[j] THEN
5           APPEND L[i] to result; i = i + 1
6       ELSE
7           APPEND R[j] to result; j = j + 1
8   APPEND all remaining values of L
9   APPEND all remaining values of R
10  RETURN result
```

Invariant

result is sorted and
contains exactly the values
already removed from L and
 R .

Cost

At most $|L| + |R| - 1$ key
comparisons.

Merge-sort pseudocode and recurrence

```
FUNCTION MERGE_SORT(A)
INPUT: array A
OUTPUT: a sorted array containing the values of A
1   IF LENGTH(A) <= 1 THEN
2       RETURN A
3   middle = LENGTH(A) DIV 2
4   L = MERGE_SORT(A[0 : middle])
5   R = MERGE_SORT(A[middle : end])
6   RETURN MERGE(L, R)
```

Running-time recurrence

$$T(n) = 2T(n/2) + \Theta(n)$$

Recursion tree

There are $\log_2 n$ levels, and each level performs $\Theta(n)$ merge work.

$$T(n) \in \Theta(n \log n)$$

Merge sort: strengths and tradeoffs

Strengths

- $\Theta(n \log n)$ in every case
- Stable with a careful merge
- Natural for linked lists
- Useful for external sorting

Tradeoffs

- Array implementation normally needs $\Theta(n)$ extra space
- Recursive calls add overhead
- Small subarrays may be faster with insertion sort

Hybrid idea

Practical implementations often use merge sort for large structure and insertion sort for small pieces.

Activity: perform one merge by hand

Sorted inputs

$$L = [1, 4, 4, 9], \quad R = [2, 4, 7, 8]$$

Record while merging

Write the selected side, output sequence, and positions of the two front pointers after every comparison.

Questions

- 1 How many comparisons are made?
- 2 Which 4 is copied first?
- 3 What condition makes the merge stable?

Quick Sort: choose a pivot and split the group

Imagine organizing people by height

Choose one person as the **pivot**. Everyone shorter moves to one side; everyone taller moves to the other side.



Then repeat

Sort the left group and the right group in exactly the same way.

Partition first, then solve smaller problems.

Quicksort: partition around a pivot

Partition goal

Rearrange the array so values smaller than the pivot are on its left and larger values are on its right. The pivot reaches its final sorted index.



Difference from merge sort

Quicksort performs the rearrangement before the recursive calls; merge sort combines results after them.

Trace one partition

Input [7, 3, 5, 2, 6, 4], choose pivot 4

Scan the remaining values and maintain a boundary between values at most the pivot and values greater than it.

Scanned value	Action	Array state
7	leave right	[7, 3, 5, 2, 6, 4]
3	move left	[3, 7, 5, 2, 6, 4]
5	leave right	[3, 7, 5, 2, 6, 4]
2	move left	[3, 2, 5, 7, 6, 4]
6	leave right	[3, 2, 5, 7, 6, 4]
finish	place pivot	[3, 2, 4, 7, 6, 5]

Quicksort pseudocode

```
FUNCTION QUICK_SORT(A, low, high)
  IF low >= high THEN RETURN
  p = PARTITION(A, low, high)
  QUICK_SORT(A, low, p - 1)
  QUICK_SORT(A, p + 1, high)

FUNCTION PARTITION(A, low, high)
  pivot = A[high]
  boundary = low
  FOR scan = low TO high - 1 DO
    IF A[scan] <= pivot THEN
      SWAP A[boundary], A[scan]
      boundary = boundary + 1
  SWAP A[boundary], A[high]
  RETURN boundary
```

Average behavior

Balanced partitions create about $\log n$ levels with $\Theta(n)$ work per level.

Worst behavior

Repeated splits of sizes 0 and $n - 1$ produce $\Theta(n^2)$ time.

Quicksort depends on pivot quality

Complexity

- Best/average: $\Theta(n \log n)$
- Worst: $\Theta(n^2)$
- Expected stack: $\Theta(\log n)$
- Worst stack: $\Theta(n)$

Pivot strategies

- Random pivot
- Median of first, middle, last
- Three-way partition for many duplicates

Practical strength

Good cache behavior and low constant factors make carefully implemented quicksort very fast for arrays.

Activity: pivot choice changes the recursion

Input

[1, 2, 3, 4, 5, 6, 7, 8]

Case A: last value as pivot

Sketch the partition sizes and recursion depth. Count the first three partition sizes.

Case B: middle value as pivot

Sketch the partition tree and compare its height with Case A.

Extension

Repeat with [4, 2, 4, 1, 4, 3, 4]. Why can a three-way partition help?

SECTION 4

Counting Sort and Radix Sort

Beyond comparison sorting

Comparison sorts

Insertion sort, bubble sort, merge sort, and quicksort learn the order by comparing two keys at a time.

“Is $A[i] \leq A[j]$?”

Lower-bound reminder

Any comparison-based sorting algorithm needs $\Omega(n \log n)$ comparisons in the worst case.

When keys are restricted

If keys are small integers, digits, or symbols from a known alphabet, we can use their values directly.

Non-comparison idea

Count how many keys occur, or sort by one digit at a time using a stable subroutine.

Counting Sort and Radix Sort exploit structure in the keys.

Counting sort: count keys, then place items

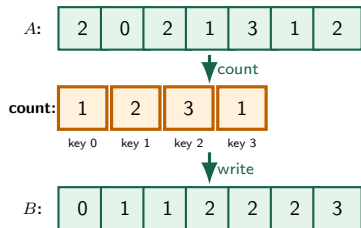
Assumption

Each key is an integer in a small known range:

$$0 \leq A[i] \leq k.$$

Core idea

- 1 Count occurrences of each key.
- 2 Convert counts into positions using prefix sums.
- 3 Place records into the output array in stable order.



Counting-sort pseudocode: stable version

```
FUNCTION COUNTING_SORT(A, k)
INPUT: array A of n integer keys in {0,...,k}
OUTPUT: stable sorted array B
1  CREATE count[0..k] initialized to 0
2  FOR i = 0 TO n - 1 DO
3      count[A[i]] = count[A[i]] + 1
4  FOR key = 1 TO k DO
5      count[key] = count[key] + count[key - 1]
6  CREATE output array B of length n
7  FOR i = n - 1 DOWNT0 0 DO
8      key = A[i]
9      B[count[key] - 1] = A[i]
10     count[key] = count[key] - 1
11 RETURN B
```

Why scan from right to left?

It preserves the relative order of equal keys, so the algorithm is **stable**.

Complexity

$$T(n, k) = \Theta(n + k),$$

$$S(n, k) = \Theta(n + k).$$

Counting sort: when is it useful?

Good situations

- Scores from 0 to 100.
- Ages, ratings, categories, small integer IDs.
- A stable subroutine inside radix sort.

Limitations

- Requires integer keys or mapped categories.
- Inefficient if k is much larger than n .
- Usually not in-place because it uses an output array.

Rule of thumb

Counting sort is attractive when $k = O(n)$, because then $\Theta(n + k) = \Theta(n)$.

Radix sort: sort by digits

Input idea

Suppose keys have d digits in base b .

329, 457, 657, 839, 436, 720, 355

Least-significant-digit strategy

- 1 Stable sort by ones digit.
- 2 Stable sort by tens digit.
- 3 Stable sort by hundreds digit.

Input	Pass 1 (ones)	Pass 2 (tens)	Pass 3 (100s)
3 2 9	7 2 0	7 2 0	3 2 9
4 5 7	3 5 5	3 2 9	3 5 5
6 5 7	4 3 6	4 3 6	4 3 6
8 3 9	4 5 7	8 3 9	4 5 7
4 3 6	6 5 7	3 5 5	6 5 7
7 2 0	3 2 9	4 5 7	7 2 0
3 5 5	8 3 9	6 5 7	8 3 9

Why stability matters

When sorting by the next digit, equal digits must keep the order already established by less significant digits.

Radix-sort pseudocode

```
FUNCTION RADIX_SORT(A, d, b)
INPUT: array A of n non-negative integer keys
OUTPUT: A sorted in non-decreasing order
1   FOR digit_position = 0 TO d - 1 DO
2       A = STABLE_COUNTING_SORT_BY_DIGIT(A,
3           digit_position, b)
4   RETURN A
```

Cost per digit

Stable counting sort by one digit costs $\Theta(n + b)$.

Total cost

$$T(n, d, b) = \Theta(d(n + b)).$$

If d and b are treated as constants, this is linear in n .

Counting sort versus Radix sort

Aspect	Counting Sort	Radix Sort
Key assumption	Keys are integers in $0..k$	Keys have d digits in base b
Main idea	Count occurrences and compute positions	Stable-sort by one digit at a time
Stability	Stable if output is filled carefully	Requires a stable digit-sorting subroutine
Time	$\Theta(n + k)$	$\Theta(d(n + b))$
Extra space	$\Theta(n + k)$	Usually $\Theta(n + b)$ per pass
Best when	Range k is small	Fixed-length integer/string keys

Common mistake

These algorithms are better only when their assumptions about the key domain are valid.

Activity: trace counting sort and radix sort

Counting-sort task

Sort

$$A = [2, 0, 2, 1, 3, 1, 2]$$

with keys in 0..3.

- 1 Build the count array.
- 2 Compute prefix sums.
- 3 Fill the output array from right to left.

Radix-sort task

Sort

$$[329, 457, 657, 839, 436, 720, 355]$$

using LSD radix sort.

- 1 Sort by ones digit.
- 2 Sort by tens digit.
- 3 Sort by hundreds digit.

SECTION 5

Comparing Sorting Algorithms

Comparison of sorting algorithms

Algorithm	Best	Average	Worst	Extra space	Important property
Insertion	n	n^2	n^2	1	Stable, adaptive, excellent for small/nearly sorted data
Bubble	n	n^2	n^2	1	Stable and simple, but performs many comparisons
Merge	$n \log n$	$n \log n$	$n \log n$	n	Stable and predictable
Quick	$n \log n$	$n \log n$	n^2	$\log n$ expected	Usually fast in-place; normally unstable
Counting	$n + k$	$n + k$	$n + k$	$n + k$	Linear when integer range k is small
Radix	$d(n + b)$	$d(n + b)$	$d(n + b)$	$n + b$	Sorts fixed-length keys by stable digit passes

Do not choose from Big- O alone

Input size, existing order, memory, record movement, stability, and worst-case guarantees can change the best choice.

FDA

Choose an algorithm for the situation

Scenarios

- 1 Twenty nearly sorted records
- 2 Millions of records with a stability requirement
- 3 Large in-memory numeric array
- 4 Scores in the range 0..100
- 5 Fixed-length student ID numbers
- 6 Data larger than available RAM

Likely choices

- 1 Insertion sort
- 2 Merge-based stable sort
- 3 Carefully implemented quicksort or a library sort
- 4 Counting sort
- 5 Radix sort
- 6 External merge sort

Discussion

Which additional facts could change each decision?

SECTION 6

Python's Built-In Sorting Functions

sorted() versus list.sort()

sorted(iterable)

```
values = [7, 3, 5, 2]
result = sorted(values)
# values is unchanged
# result is [2, 3, 5, 7]
```

Returns a new list and accepts any iterable.

list.sort()

```
values = [7, 3, 5, 2]
answer = values.sort()
# values is now sorted
# answer is None
```

Modifies the list in place and returns None.

Sort records with key and reverse

```
students = [  
    {"id": "S1", "name": "Lan", "gpa": 3.4},  
    {"id": "S2", "name": "Minh", "gpa": 3.8},  
    {"id": "S3", "name": "An", "gpa": 3.8},  
]  
  
ranked = sorted(students,  
                 key=lambda s: (-s["gpa"], s["name"]))
```

Key function

Converts each record into the value or tuple used for comparison.

Python guarantee

Python sorting is stable, so equal-key records retain their previous relative order.

Common sorting mistakes

Mistakes

- Expecting `list.sort()` to return the list
- Sorting numbers stored as strings
- Writing a comparison key with inconsistent types
- Forgetting whether the original list may be modified

Good workflow

- 1 Define the required order precisely.
- 2 Choose the key and direction.
- 3 Decide whether a copy is needed.
- 4 Test duplicates and boundary cases.

Integrated activity: sort transaction records

Task

Sort transactions by date ascending; for equal dates, sort amount descending; preserve input order when both keys are equal.

```
transactions = [  
    {"date": "2026-08-10", "amount": 250, "id": "T1"},  
    {"date": "2026-08-09", "amount": 400, "id": "T2"},  
    {"date": "2026-08-10", "amount": 300, "id": "T3"},  
    {"date": "2026-08-10", "amount": 250, "id": "T4"},  
]
```

Student questions

Write the key function, predict the output IDs, and explain where stability matters.

Key takeaways

- Sorting requires both an ordering rule and preservation of all records.
- Insertion sort is adaptive and effective for small or nearly sorted input.
- Merge sort guarantees $\Theta(n \log n)$ time and is naturally stable.
- Bubble sort is easy to understand but inefficient for general use.
- Quicksort is usually fast, but poor pivots can cause quadratic time.
- Counting sort and radix sort can be linear when keys have restricted structure.
- Python's stable built-in sort should be the default in application code; studying algorithms explains its tradeoffs and behavior.

After-class practice

- 1 Trace all four algorithms on $[5, 1, 4, 2, 8, 2]$.
- 2 Count insertion-sort shifts for sorted and reverse-sorted arrays.
- 3 Modify merge sort to order records by a supplied key function.
- 4 Show an input that makes last-element-pivot quicksort quadratic.
- 5 Trace counting sort on $[2, 0, 2, 1, 3, 1, 2]$.
- 6 Trace one LSD radix-sort pass for three-digit numbers.
- 7 Use Python to sort student records by class, descending GPA, and name.